

Question 2 (Option – A)

Optimizing EV Charging Cost Using Penalty Method

Team Members:

- P Harshavardhan Reddy (BT2024064)
 - N V Pradyumna (BT2024157)
 - Naga Rutwik Malapaka (BT2024091)
-

Introduction to the Problem

Electric Vehicle (EV) charging optimization aims to allocate charging amounts across multiple vehicles to minimize total energy cost while satisfying operational and system constraints. Uncontrolled or poorly scheduled charging can lead to high costs, excessive peak demand, or violations of individual vehicle limits.

In this project, we model EV charging cost using a **quartic cost function** that captures nonlinear energy pricing and interactions between vehicles. The task includes:

- **Unconstrained Optimization** – minimizing the total cost ignoring constraints.
- **Constrained Optimization** – enforcing realistic energy, power, and vehicle limits.

This report (Part 2) presents the **Constrained Optimization Method**, covering the **mathematical formulation, penalty construction, algorithm steps, and implementation details**.

Constrained EV Charging Optimization

Real EV charging schedules must satisfy practical constraints:

- **Total Energy Requirement:**

$$\sum_{i=1}^n x_i = E_{\text{total}}$$

- **Maximum Total Power:**

$$\sum_{i=1}^n x_i^2 \leq P_{\text{max}}$$

- **Individual Charging Limits:**

$$x_{\min} \leq x_i \leq x_{\max}$$

The overall optimization problem becomes:

$$\begin{aligned} \min_x \quad & f(x) = x^T Q x + \sum_i \alpha_i x_i^3 + \sum_i \beta_i x_i^4 \\ \text{s.t.} \quad & \sum_i x_i = E_{\text{total}}, \sum_i x_i^2 \leq P_{\max}, x_{\min} \leq x_i \leq x_{\max} \end{aligned}$$

Such constraints make the problem significantly more complex. To solve it, we implement the **Penalty Method**.

Penalty Method Approach

The **Penalty Method** transforms the constrained problem into an unconstrained one by adding penalty terms for constraint violations:

$$\begin{aligned} P(x, \rho) = f(x) + \frac{\rho}{2} (\sum_i x_i - E_{\text{total}})^2 + \frac{\rho}{2} \max(0, \sum_i x_i^2 - P_{\max})^2 + \frac{\rho}{2} \sum_i \max(0, x_{\min} \\ - x_i)^2 + \frac{\rho}{2} \sum_i \max(0, x_i - x_{\max})^2 \end{aligned}$$

Where:

- ρ = penalty parameter
- As $\rho \rightarrow \infty$, constraint violations become extremely costly.
- Minimizing $P(x, \rho)$ pushes the solution into the feasible region.

Algorithm: Penalty Method + Gradient Descent

The method alternates between:

- **Outer loop:** increase penalty if constraints are violated
- **Inner loop:** minimize the penalized function using gradient descent with backtracking line search

Outer Loop Steps

1. Start with small penalty: $\rho_0 = 10^5$
2. Solve the unconstrained problem with gradient descent
3. Measure violations:
 - $|\sum_i x_i - E_{\text{total}}|$
 - $\max(0, \sum_i x_i^2 - P_{\text{max}})$
 - $\sum_i \max(0, x_{\text{min}} - x_i)^2$ and $\sum_i \max(0, x_i - x_{\text{max}})^2$
4. If all violations < tolerance $\rightarrow$ stop
5. Otherwise, increase penalty: $\rho_{k+1} = 2\rho_k$

The penalty becomes stronger at each iteration, forcing feasibility.

Inner Loop: Gradient Descent with Penalty Terms

For a fixed ρ , the gradient of the penalized function is:

$$\begin{aligned} \nabla P(x, \rho) = \nabla f(x) + \rho \left(\sum_i x_i - E_{\text{total}} \right) \mathbf{1} + \rho \max(0, \sum_i x_i^2 - P_{\text{max}}) (2x) \\ + \rho \sum_i \text{gradient of bound violations} \end{aligned}$$

Backtracking line search ensures **stable updates**, even with steep penalties.

Advantages of the Penalty Method

- Handles **multiple constraints simultaneously**
 - Does **not require Lagrange multipliers**
 - Converts the problem into **repeated unconstrained optimization** (easy to implement)
 - Guarantees **feasibility** as ρ increases
-

Limitations

- Very large ρ makes the optimization landscape steep
- Convergence can be slower

- Sensitive to penalty growth schedule

Despite these limitations, the method successfully returns:

- Charging values satisfying total energy
- Peak power within limits
- Individual EV charging bounds.

Short Interpretation About Solution Behaviour

At the start of the iterations, the optimizer moves aggressively in the direction of reducing the original cost function $f(x)$. However, this often violates constraints, especially the total-energy equality and power limit. As the penalty terms become dominant, the search gradually shifts its priority from minimizing cost to correcting infeasibility. This results in the solution being "pulled" back toward the feasible region. An observation is that large penalty parameters make the optimization landscape very steep, causing smaller step sizes (due to backtracking) and slower convergence. This reflects the fundamental trade-off in penalty methods:

- Small penalty (ρ) $\rightarrow$ faster descent but more constraint violation.
- Large penalty $\rightarrow$ better feasibility but slower, more unstable updates.

Overall, the solution behaviour highlights that feasibility is achieved at the cost of slightly higher objective value, which is expected in practical EV charging systems where operational constraints override purely cost-optimal behaviour. The method therefore provides a realistic balance between minimizing energy cost and maintaining system-level safety limits.

In our case, the algorithm finally converges to a point where:

- The total charging energy matches the requirement.
- The quadratic power constraint is satisfied.
- Individual EV charging limits are respected.
- The achieved $f(x)$ is close to the unconstrained minimum, but slightly higher due to constraints forcing a more balanced allocation.

Output

Iter	f(x)	P(x)	grad_P
1	6475.766568	290091171.945609	376755944.450290
2	54.378714	143115580.966837	17284277.835845
3	2922.418233	74885.368186	379056.981395
4	3157.606819	62194.223486	343950.421717
5	2943.368676	51744.987775	312095.285166
6	3136.990872	43141.597230	283190.430046
7	2960.600661	36057.993722	256962.622492
8	3119.991002	30225.688844	233163.920074
9	2974.751085	25423.645727	211569.355523
10	3105.951416	21469.857930	191974.787385
11	2986.350800	18214.483433	174194.988924
12	3094.337633	15534.141118	158061.881762
13	2995.841259	13327.252467	143422.959672
14	3084.713293	11510.178022	130139.836486
15	3003.588902	10014.057373	118086.946492
16	3076.721636	8782.191675	107150.350376
17	3009.897445	7767.900279	97226.663955
18	3070.070628	6932.747234	88222.076076
19	3015.018312	6245.087636	80051.465749
20	3064.520985	5678.865867	72637.594243
21	3019.159431	5212.629798	65910.377603
22	3059.876479	4828.716393	59806.221936
23	3022.492623	4512.583228	54267.424062
24	3055.976083	4252.256569	49241.624544
25	3025.159775	4037.878095	44681.313952
26	3052.687579	3861.330836	40543.382741
27	3027.277991	3715.931828	36788.714560
28	3049.902356	3596.178592	33381.815792
29	3028.943883	3497.540745	30290.480596
30	3047.531158	3416.288163	27485.485991
31	3030.237120	3349.349700	24940.315992
32	3045.500611	3294.196709	22630.910632
33	3031.223387	3248.747259	20535.438782

33	3031.223387	3248.747259	20535.438782
34	3043.750384	3211.287185	18634.091560
35	3031.956821	3180.405146	16908.895255
36	3042.230856	3154.939110	15343.541267
37	3032.482035	3133.932331	13923.232064
38	3040.901212	3116.597067	12634.541196
39	3032.835788	3102.284740	11465.286453
40	3039.727869	3090.461342	10404.414636
41	3033.048360	3080.687202	9441.897126
42	3038.683197	3072.600316	8568.635027
43	3033.144695	3065.902630	7776.373193
44	3037.744458	3060.348742	7057.622152
45	3033.145338	3055.736604	6405.587329
46	3036.892940	3051.899856	5814.104783
47	3033.067208	3048.701529	5277.582947
48	3036.113242	3046.028849	4790.949741
49	3032.924239	3043.788965	4349.604620
50	3035.392689	3041.905426	3949.375045
51	3032.727906	3040.315291	3586.477019
52	3034.720846	3038.966732	3257.479260
53	3032.487657	3037.817080	2959.270717
54	3034.089125	3036.831204	2689.031076
55	3032.211273	3035.980178	2444.204003
56	3033.490457	3035.240184	2222.472843
57	3031.905160	3034.591616	2021.738546
58	3032.919021	3034.018327	1840.099600
59	3031.574593	3033.507025	1675.833763
60	3032.370028	3033.046766	1527.381407
61	3031.223915	3032.628538	1393.330296
62	3031.839532	3032.244922	1272.401638
63	3030.856701	3031.889809	1163.437246
64	3031.324288	3031.558169	1065.387676
65	3030.475896	3031.245862	977.301213
66	3030.821622	3030.949478	898.313596
67	3030.083923	3030.666210	827.638390
68	3030.329334	3030.393747	764.557975

68	3030.329334	3030.393747	764.557975
69	3029.682779	3030.130185	708.415124
70	3029.845613	3029.873957	658.605217
71	3029.274106	3029.623774	614.569175
72	3029.368967	3029.378573	575.787238
73	3028.859258	3029.137481	541.773717
74	3028.898168	3028.899776	512.072863
75	3028.439348	3028.664865	486.255909
76	3028.425071	3028.636418	1033.198750
77	3027.598416	3028.330472	948.399320
78	3027.925209	3028.039399	872.413041
79	3027.205479	3027.760577	804.478729
80	3027.435375	3027.491846	743.901000
81	3026.803687	3027.231431	690.042068
82	3026.953821	3026.977867	642.314204
83	3026.394627	3026.729951	600.172947
84	3026.479104	3026.486689	563.111204
85	3025.979605	3026.247266	530.654382
86	3026.010039	3026.011008	502.356650
87	3025.559698	3025.777363	477.798397
88	3025.531612	3025.722270	1002.197834
89	3024.725968	3025.422291	920.581642
90	3025.034485	3025.136204	847.502161
91	3024.332126	3024.861560	782.222375
92	3024.547049	3024.596343	724.068137
93	3023.929734	3024.338893	672.420183
94	3024.067613	3024.087844	626.706848
95	3023.520326	3023.842070	586.397567
96	3023.594787	3023.600646	550.997324
97	3023.105164	3023.362808	520.042158
98	3023.127425	3023.127929	493.095847
99	3022.243189	3023.125698	1059.931991
100	3022.643888	3022.815567	972.347216

FINAL x^* =

[6.37536984 6.19584615 5.49468709 5.64141641 6.09726804 5.82332686
5.88739433 6.24604537 5.86295565 6.37204982]

Final $f(x^*)$ after penalty method = 3021.858551804131

sum x_i = 59.996359569525936

sum x_i^2 = 360.7905914557152

□